

RESEARCH ARTICLE

Optimizing Traffic Speed Prediction Using a Multi-Objective Genetic Algorithm-Enhanced RNN for Intelligent Transportation Systems

C. SWETHA PRIYA  **AND F. SAGAYARAJ FRANCIS**

Department of Computer Science and Engineering, Puducherry Technological University, Puducherry 605014, India

Corresponding author: C. Swetha Priya (swethapriyataru@gmail.com)

ABSTRACT Over the past decade, major cities have faced significant traffic congestion, accidents, and pollution due to increased vehicle usage, urbanization, and migration. An Intelligent Transportation System (ITS) can enhance transportation planning and alleviate congestion. ITS utilizes traffic prediction models to help prevent traffic bottlenecks, improve mobility and safety, and reduce environmental impacts. However, developing these models involves several challenges, including understanding spatiotemporal nonlinearities, making accurate predictions, minimizing prediction time, and reducing model complexity. Many existing approaches integrate Convolutional Neural Networks (CNNs) and variants of Recurrent Neural Networks (RNNs) to analyze spatially correlated traffic data over time. Nevertheless, these hybrid models often require significant storage space, contain numerous learnable parameters, and involve extensive training, validation, and testing times. To address these challenges, we propose a novel methodology that combines a genetic algorithm (GA) with Random Forest Cross-Validation (RF-CV) to evaluate input features and select the most relevant subset. Additionally, we developed a Multi-Objective Genetic Algorithm (MOGA)-enhanced RNN model to optimize hyperparameters and achieve accurate traffic speed predictions. Our proposed methodology balances the trade-offs between prediction accuracy, model size, and computational efficiency by identifying an optimal set of relevant features and hyperparameters. We evaluated our model using the Performance Measurement System (PeMS)-10 dataset and compared its performance against baseline and advanced models from existing literature. Our model achieved a Mean Absolute Error (MAE) of 0.028993, an R^2 score of 0.999490, and training, validation, and testing times of 81.64 seconds, 0.15 seconds, and 0.18 seconds, respectively. Additionally, the model size was 203,118 bytes, with 14,617 parameters. A comprehensive comparative study demonstrates that our approach outperforms state-of-the-art models in both prediction accuracy and computational efficiency.

INDEX TERMS Deep learning models, genetic algorithms, features, hyperparameters, accuracy, inference time, model size, vehicle manoeuvrability.

I. INTRODUCTION

Over the last decade, major cities worldwide have faced significant challenges related to vehicle traffic congestion [1]. This issue is characterized by longer vehicle queues, extended travel times, and reduced travel speeds. The primary causes include high vehicle density due to road bottlenecks, inef-

ficient traffic signal management, and adverse weather conditions. Consequently, traffic congestion results in increased pollution, wasted time and fuel, frequent accidents, and substantial economic and environmental impacts.

According to the Urban Mobility Report 2023, managing congestion in 494 urban areas across the United States incurs an estimated annual cost of \$101 billion. This situation results in approximately 4.3 billion hours of travel delays, 1.7 billion gallons of fuel wasted, and the emission of 18 million tons of

The associate editor coordinating the review of this manuscript and approving it for publication was Prakasam Periasamy .

CO_2 [2]. The report further predicts that by 2025, the costs of congestion mitigation could rise to \$237 billion, with travel delays increasing to 10 billion hours and 3.6 billion gallons of fuel wasted by 2025. To address these challenges, Intelligent Transportation Systems (ITS) offer promising solutions to improve transportation planning and alleviate congestion. ITS leverages advanced traffic parameter prediction models to provide valuable insights into traffic patterns, travel times, and congestion levels. Consequently, the accurate prediction of these traffic parameters remains a critical and complex area of research.

A. CHALLENGES

Traffic prediction models are broadly categorized into short-term and long-term models [3]. Short-term traffic prediction focuses on forecasting imminent traffic conditions within a specific region, including parameters such as traffic flow, speed, occupancy rate, and congestion, over a time horizon ranging from a few minutes to a few hours. In contrast, long-term traffic prediction involves forecasting traffic conditions over extended periods, ranging from several hours to days, weeks, or even months. These models face several challenges, such as understanding spatiotemporal nonlinearities, ensuring prediction accuracy, reducing inference times, and managing model complexity. To address these challenges, various parametric, nonparametric, and hybrid models have been developed [4]. Parametric models, such as Autoregressive Integrated Moving Average (ARIMA) and its variants [5], [6], [7], are widely used for time-series analysis. These models excel at learning temporal patterns from historical traffic data to predict future traffic conditions. However, their linear nature makes them struggle to capture nonlinear relationships or complex dynamics in traffic systems.

In response, extensive research has shifted toward non-parametric models like Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Artificial Neural Networks (ANN), and Deep Neural Networks (DNN), which can address the nonlinear complexities inherent in traffic data [8], [9]. More advanced models, such as RNN, Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU), have been applied to learn intricate temporal patterns in sequential data. To further capture spatiotemporal dependencies in traffic data, CNN has been utilized, leading to the development of hybrid models such as Conv-LSTM [4], CNN-GRU [11], and Graph Convolutional Networks (GCN) [12]. Although hybrid models achieve high prediction accuracy, they face two significant challenges: 1. Systematically identifying an optimal or near-optimal subset of relevant features. 2. Managing high time-to-accuracy and model complexity due to the absence of automated approaches for hyperparameter tuning.

It is widely acknowledged that selecting the best subset of relevant features and hyperparameters has a profound impact on the performance of deep learning models. Consequently,

researchers have increasingly focused on automating the parameter selection process to replace random or heuristic settings. Automation allows for a comprehensive exploration and evaluation of the search space for feature subsets and hyperparameters. Automated techniques optimize various aspects of predictive models, including network architecture, learnable parameters, inference time, and overall model complexity. Common algorithms for this purpose include Grid Search, Genetic Algorithms, Particle Swarm Optimization, and Equilibrium Optimization [13], [14], [15], [16]. These methods have shown positive outcomes in many Internet of Things (IoT) and ITS data prediction scenarios.

However, heuristic optimization algorithms often face challenges such as high convergence times, exponential search spaces for feature subsets, and susceptibility to local optima. Studies [13], [14], [15], [16], [17], [18] aimed at improving prediction performance through hyperparameter optimization have demonstrated success but are prone to overfitting when applied to larger, real-time datasets. Therefore, these approaches must strike a balance between accuracy, efficiency, and computational complexity. Our proposed approach is the first multi-objective GA-based method for real-time PeMS traffic speed prediction. This method enables the development of a more lightweight and generalized RNN model, which is less susceptible to overfitting and better suited for real-time traffic forecasting applications.

Optimizing traffic speed is a multifaceted challenge that extends beyond traffic signal timing and road design. Vehicle manoeuvrability significantly influences traffic flow dynamics. For instance, frequent lane changes, often caused by poor lane discipline, disrupt traffic patterns and reduce flow velocity. This interdependence highlights the need for a holistic approach to traffic speed optimization. Studies by Guo et al. [19] and Akopov et al. [20] underscore the importance of improving vehicle manoeuvrability and optimizing speed across diverse transportation systems. These insights demonstrate that addressing traffic dynamics holistically is essential for improving overall traffic efficiency and mobility.

B. PROBLEM IDENTIFICATION

We collected a real-time traffic dataset from the PeMS of the California Department of Transportation for our experiment. The dataset includes traffic flow, speed, and occupancy rate data from five Vehicle Detector Stations (VDS). We initially made one-hour-ahead predictions using basic and advanced models, including RNN, GRU, LSTM, Bi-LSTM, LSTM-AM, Conv-LSTM, CNN-LSTM, and CNN-GRU, with a sequence length of 24-time steps. Models like CNN-LSTM and CNN-GRU use Conv1D layers with filters to capture inter-feature dependencies in the input data across time steps. These filters act as feature extractors, learning patterns across features such as flow, speed, and occupancy at each time step. By employing a kernel size of 1, these filters focus

TABLE 1. Training time, Test time and R^2 for flow prediction.

	VDS1			VDS2			VDS3			VDS4			VDS5		
	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2
RNN	4.33	0.51	0.969	4.43	0.42	0.968	4.361	0.43	0.962	4.12	0.45	0.957	6.50	20.25	0.963
GRU	7.80	0.71	0.975	7.67	0.70	0.962	7.84	0.74	0.964	8.00	0.74	0.972	11.88	52.11	0.968
LSTM	7.87	0.82	0.973	7.73	0.83	0.968	7.53	0.81	0.968	7.56	0.81	0.963	11.59	1.71	0.964
Bi-LSTM	9.20	1.21	0.969	9.29	1.17	0.972	9.28	1.17	0.965	8.95	1.17	0.967	16.94	40.08	0.952
LSTM-AM	8.99	0.92	0.960	8.93	0.91	0.965	8.92	0.88	0.938	8.99	0.91	0.964	14.03	1.71	0.965
Conv-LSTM	21.53	1.35	0.967	21.57	1.30	0.971	21.36	1.28	0.962	21.40	1.27	0.970	41.67	14.75	0.966
CNN-LSTM	14.69	1.40	0.972	14.62	1.29	0.973	14.59	1.30	0.968	14.75	1.32	0.963	40.27	33.70	0.964
CNN-GRU	13.73	1.18	0.976	13.81	1.14	0.973	13.78	1.14	0.966	13.84	1.13	0.959	36.30	37.84	0.971

TABLE 2. Training time, Test time and R^2 for occupancy rate prediction.

	VDS1			VDS2			VDS3			VDS4			VDS5		
	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2
RNN	4.47	0.49	0.947	4.46	0.44	0.918	4.39	0.44	0.771	4.45	0.43	0.834	4.38	0.43	0.917
GRU	8.31	1.20	0.955	7.56	0.71	0.948	7.93	0.79	0.777	7.98	0.76	0.880	8.13	0.70	0.941
LSTM	7.95	0.87	0.944	8.06	0.84	0.933	7.62	0.85	0.760	8.13	0.87	0.897	8.13	0.90	0.941
Bi-LSTM	9.33	1.15	0.956	9.21	1.14	0.952	9.55	1.18	0.795	9.49	1.17	0.864	9.42	1.13	0.936
LSTM-AM	9.24	0.92	0.950	9.32	0.90	0.936	9.05	0.95	0.758	9.39	0.93	0.691	9.37	0.95	0.923
Conv-LSTM	21.28	1.31	0.960	21.72	1.36	0.939	21.5	1.33	0.764	21.62	1.27	0.683	21.69	1.35	0.939
CNN-LSTM	14.58	1.31	0.961	14.75	1.37	0.936	14.68	1.34	0.786	14.78	1.30	0.893	14.81	1.31	0.940
CNN-GRU	13.74	1.15	0.953	13.98	1.16	0.885	13.84	1.11	0.782	13.98	1.14	0.897	13.92	1.15	0.938

TABLE 3. Training time(Tr-T), Test time(Ts-T) and R^2 for speed prediction.

	VDS1			VDS2			VDS3			VDS4			VDS5		
	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2	Tr-T	Ts-T	R^2
RNN	4.41	0.51	0.760	7.24	6.79	0.734	2.60	0.43	0.586	6.82	0.54	0.530	6.97	0.84	0.715
GRU	8.01	0.76	0.797	12.60	61.16	0.729	4.67	0.76	0.545	12.71	1.84	0.515	12.71	26.92	0.718
LSTM	7.92	0.82	0.783	11.98	41.75	0.742	4.60	1.05	0.608	12.40	1.29	0.565	12.85	17.31	0.716
Bi-LSTM	9.42	1.16	0.776	19.42	61.36	0.766	8.52	1.41	0.581	19.71	2.68	0.393	20.59	2.04	0.682
LSTM-AM	9.13	0.94	0.788	14.46	1.58	0.701	5.93	55.19	0.07	15.13	49.04	0.386	15.11	1.80	0.703
Conv-LSTM	21.37	1.28	0.776	47.29	38.12	0.765	34.18	58.72	0.589	46.51	35.65	0.514	50.31	41.93	0.725
CNN-LSTM	14.44	1.30	0.796	44.48	45.32	0.770	22.11	60.13	0.594	45.19	49.34	0.548	45.36	3.10	0.733
CNN-GRU	13.58	1.15	0.799	38.70	33.78	0.767	18.13	36.22	0.505	40.53	1.87	0.567	40.37	2.03	0.705

on interactions between individual features while preserving the original feature resolution. This approach ensures that important relationships between features are captured before the data is passed to the LSTM or GRU layers for temporal modelling.

Tables 1, 2, and 3 present the Training time(Tr-T), Testing time(Ts-T) and coefficient of determination (R^2) for predicting traffic flow, occupancy rate, and speed at different VDS. The flow and occupancy rate predictions exhibit acceptable accuracy, indicating that the state-of-the-art foundation models effectively captured a significant portion of the variance in the data. The speed predictions,

especially at VDS3, showed low R^2 values. The trends and correlations in Figures 1 and 2 show that speed parameters have a very low negative correlation with flow and occupancy parameters, indicating that flow and occupancy have a limited linear correlation with speed.

Additionally, speed variables have high autocorrelation, which can cause models to overfit and not generalize well to new data. The PeMS data descriptive statistics in Table 4 indicate that the speed data at VDS3 have a low standard deviation, which means most values are close to the average with little variation. Despite the low standard deviation, there are many instances where the actual values differ significantly

TABLE 4. Statistical summary of the PeMS dataset across VDS segments.

	VDS1			VDS2			VDS3			VDS4			VDS5		
	f^{s1}	S^{s1}	O^{s1}	f^{s2}	S^{s2}	O^{s2}	f^{s3}	S^{s3}	O^{s3}	f^{s4}	S^{s4}	O^{s4}	f^{s5}	S^{s5}	O^{s5}
Count	19678	19678	19678	19678	19678	19678	19678	19678	19678	19678	19678	19678	19678	19678	19678
Mean	1001.2	65.6	3.5	1123	65	3.2	1104.9	65.2	2.6	1237.7	64.9	3.2	1243.5	63.1	3.6
Std	521.3	2.5	1.83	578.7	3.3	1.80	562.2	3.05	1.58	625.8	3.0	1.73	547	1.52	1.42
Min	104	49.6	0.2	115	41.6	0.3	100	24.6	0.2	104	28.9	0.2	173	55.7	0.5
25%	486	64.1	2	567	63.4	1.6	560	64.2	1.4	621	63.8	1.6	150	62.5	2.5
50%	1053	6.6	3.3	1162	65.9	3.1	1136	65.7	2.6	1296	65.3	3.2	1250	63.3	3.5
75%	1439	67.6	4.8	1606	67.3	4.5	1585	67.1	3.6	1773	66.6	4.5	1664	63.9	4.6
Max	2170	74.9	10.3	2868	75.8	24	2878	75.8	35.2	3112	75	48.8	3170	6803	10.5

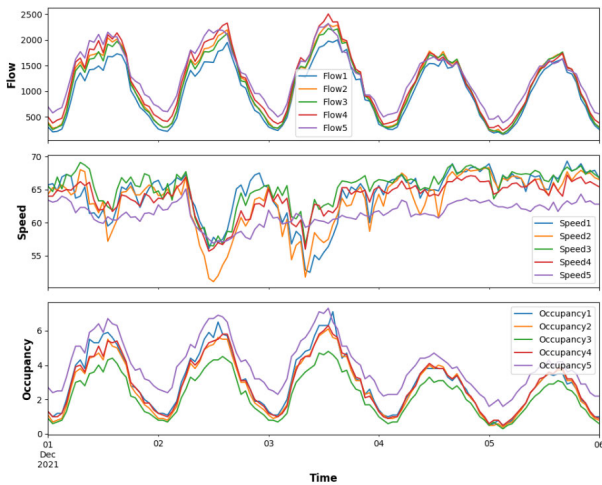


FIGURE 1. Trend of PeMS dataset traffic parameters.

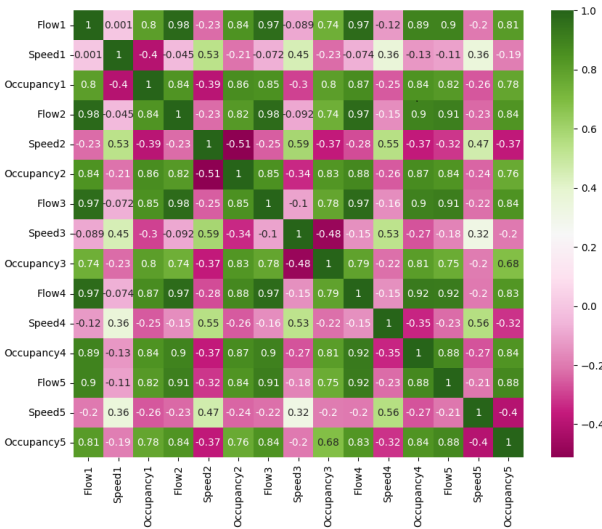


FIGURE 2. PeMS correlation matrix.

from the average, especially in the sharp dips shown in Figure 3. The state-of-the-art foundation models capture only

the general trend of the speed data to some extent but not the outliers of the actual speed variations.

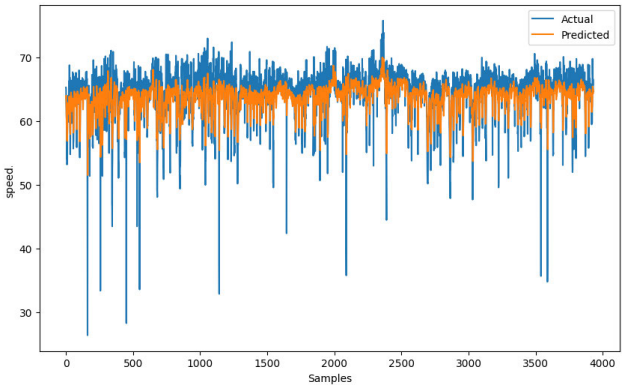


FIGURE 3. Actual vs Predicted speed values at VDS-3.

In Figure 2, the Pearson correlation matrix shows that the speed variables correlate low to medium with flow, occupancy and neighbour speed features. This suggests that the speed data instances contain complex nonlinear spatiotemporal dependencies. Therefore, it is crucial to identify a relevant subset of features from the original input features that exhibit subtle variations. Additionally, a model must generalize well to unseen data without being overly complex.

Figure 4 illustrates the number of parameters used in the basic and state-of-the-art foundation models for PeMS traffic speed predictions. A higher number of parameters indicates a more complicated and computationally expensive model, potentially making it slower to train and test the data samples. Among the models in Figure 4, CNN-LSTM and CNN-GRU employ the highest parameters. At the same time, RNN utilizes the fewest parameters, making it the simplest and least complex model despite other models showing only marginal improvements in R^2 compared to RNN, as presented in Table 4. The RNN provides the best trade-off between test time, number of parameters, and predictive accuracy. To improve the RNN's overall

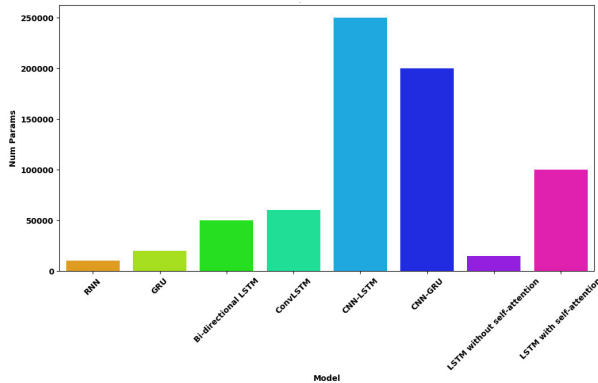


FIGURE 4. Number of parameters across different models.

performance, we utilized a genetic algorithm to select the best subset of input features by correlating spatiotemporal data of neighbour VDS. Additionally, we employed a multi-objective genetic algorithm to perform hyperparameter optimization of the RNN model, aiming to enhance performance in terms of accuracy, test time, and model complexity while improving the stability of RNNs, particularly in avoiding the vanishing and exploding gradient problems. Moreover, most existing literature [13], [14], [15], [16], [17], [18] has focused on a single objective: improving the model's accuracy using GA-based parameter optimization. High-performing deep learning models often become more complex and deeper, leading to longer training and test times, a larger number of parameters, and increased model size.

C. OUR CONTRIBUTIONS

To tackle the above issues, we have established multiple competing objectives: reducing MSE, RMSE, training and test times, model size, and maximizing accuracy in the MOGA fitness function. This research is the first to propose a MOGA-enhanced RNN framework for real-time PeMS traffic data, resulting in a lightweight, more generalized model that is less prone to overfitting. The key contributions include:

- 1) **Relevant Feature Selection:** We have chosen the optimal or nearly optimal subset of input features that capture subtle and significant variations in the speed data. We used a combination of RF-CV with a genetic algorithm to achieve this. The random forest cross-validation estimates the aggregated MSE (AMSE) and accuracy on the validation set. A genetic algorithm explores and selects the optimal or nearly optimal feature subset, aiming for lower AMSE and higher accuracy as the objective function.
- 2) **Hyperparameter Optimization:** Our approach combines the RNN model with a multi-objective Non-dominated Sorting Genetic Algorithm 2 (NSGA-II) to minimize MSE, Root Mean Squared Error (RMSE), training and test times, model size, and maximize

R^2 . Additionally, hyperparameter optimization also controls the flow of gradients using regularization methods, further balancing the model's predictive performance and computational efficiency.

- 3) **Comprehensive Model Evaluation and Comparison:** To select the best-performing model, we will evaluate and compare basic and advanced models such as RNN, GRU, LSTM, Bi-GRU, Bi-LSTM, LSTM-AM, Conv-LSTM, CNN-LSTM, and CNN-GRU. Our unique comparison criteria will include not only accuracy metrics (MAE, MSE, RMSE, MAPE, R^2) but also efficiency metrics (Training Time, Validation Time, Test Time) and complexity metrics (Model Size, Number of Parameters).

The rest of the article is organized as follows. Section II introduces the related works. In Section III, the problem formulation is presented. In Section IV, the theoretical background is discussed. Section V describes the methodology. Experimental results and analysis are presented in Section VI. Section VII provides a brief conclusion and future directions.

II. RELATED WORKS

Over the past several decades, significant research has been conducted on vehicle traffic prediction. The fundamental theory of traffic prediction, focusing on creating models to describe the relationships between flow, speed, and density, was a major area of activity in the 1950s and 1960s. In 1955, Lighthill and Whitham introduced the Lighthill-Whitham-Richards (LWR) traffic flow model, which laid the foundation for macroscopic traffic modelling. Edie's work in 1961 on car-following models [21] represents one of the early contributions to microscopic traffic modelling.

In 1970, Box and Jenkins introduced the ARIMA model in their seminal work [22], which has since been extensively used in traffic forecasting, even though it was not originally designed for transportation applications. Following the inception of ARIMA, various extensions were developed to address its limitations. For instance, Lee et al. [5] proposed a subset ARIMA model for short-term traffic volume forecasting, which provides more stable and accurate forecasts than the full ARIMA model. Williams et al. [7] established the theoretical foundation for modelling univariate traffic condition data streams as seasonal ARIMA processes, leveraging the Wold decomposition theorem. Efforts were also directed toward multivariate forecast models, including upstream sensor data, to predict traffic at downstream locations. For example, Williams et al. [6] employed the ARIMAX model for such tasks.

Hybrid models have been extensively explored, combining ARIMA with machine learning and neural networks. Miao et al. [23] developed a hybrid model named ESARIMA by integrating ARIMA with singular value decomposition (SVD) and empirical mode decomposition (EMD) techniques for improved traffic flow prediction. However, the linear nature of ARIMA-based models limits their ability to capture

the nonlinear relationships and complex dynamics inherent in traffic systems. To overcome these limitations, neural networks have been widely adopted due to their ability to model nonlinearity, extract features, handle large datasets, and achieve high predictive accuracy. Zheng et al. [24] proposed a hybrid neural network model that combines predictions from backpropagation (BP) and radial basis function (RBF) neural networks for traffic flow prediction. Stacked autoencoders have also shown promise for their nonlinear learning capabilities and compact representation in traffic flow prediction, as demonstrated by Lv et al. [25], Duan et al. [26], and Yang et al. [27].

Given the significant variations in traffic patterns over time, RNNs and their advanced variants, such as LSTM and GRU, have become the most widely accepted models for capturing temporal dependencies in traffic data. For instance, Zhao et al. [28], Jia et al. [29], and Abbas et al. [30] demonstrated that LSTM models can effectively capture long-term trends and patterns in traffic data, improving the prediction of flow, speed, and density, respectively. Compared to LSTM, GRU is computationally more efficient and simpler, making it advantageous in certain scenarios. For example, Zhang et al. [31] employed a GRU-based approach to predict traffic flow while incorporating weather condition data.

The accurate representation of traffic dynamics and adaptability to different scales, ranging from individual intersections to entire city networks, requires simultaneous analysis of spatial and temporal features. This has led to the development of hybrid models for spatiotemporal analysis. Cui et al. [32] predicted network-level traffic speeds using bidirectional LSTM, capturing spatial and temporal dependencies. City-wide crowd flow forecasting has been addressed by Zhang et al. [33], [34] using the Spatial-Temporal Residual Network (ST-RESNET), which effectively analyzes the spatiotemporal properties of traffic networks. Yu et al. [35] developed spatiotemporal recurrent convolutional networks (SRCNs), where spatial dependencies are captured using convolutional neural networks (CNNs) on network-wide traffic speeds represented as static images, and temporal dynamics are captured using LSTM. Recognizing the influence of adjacent road sections on traffic flow, Yang et al. [36] devised spatiotemporal correlation methods, providing input for ConvLSTM models. Ma et al. [37] proposed a framework combining a Capsule Network (CapsNet) for spatial feature extraction and a Nested LSTM (NLSTM) for capturing hierarchical temporal dependencies. Similarly, Ma et al. [11] developed a hybrid model using CNN and bidirectional GRU, where spatial correlations were modelled using a feature selection algorithm. Cheng et al. [38] proposed a CNN-LSTM hybrid model, where the predictable relationship of variables was first determined using the Vector Auto-Regression (VAR) model.

Hybrid approaches, such as those involving graph convolutional networks (GCNs) integrated with RNNs or their

variants, have gained importance for effectively capturing complex spatial dependencies in large-scale traffic networks [39], [40], [41], [42], [43], [44], [45]. Hyperparameter tuning is critical for optimizing the performance of deep learning models in traffic prediction. To enhance accuracy and reduce training time, researchers such as Puentes et al. [46], Mao et al. [47], Zhang et al. [48], and Niaraki et al. [49] have explored hyperparameter optimization using evolutionary algorithms in conjunction with RNN variants. Recent studies have also focused on optimizing vehicle manoeuvrability to alleviate traffic congestion. Guo et al. [19] and Akopov et al. [20] examined parameters such as speed planning, energy management, state-of-charge (SOC), traffic density estimation, and lane-changing behaviours.

The evolution of traffic prediction models, from basic statistical methods to advanced hybrid machine learning techniques, highlights significant advancements in accuracy and scalability. The current research trend includes developing hybrid models, automated hyperparameter tuning using evolutionary algorithms, and optimization of vehicle manoeuvrability. However, challenges remain in balancing prediction accuracy and computational efficiency. We propose a comprehensive and systematic approach to traffic prediction aimed at developing a generalized, robust, accurate, and computationally efficient model.

III. PROBLEM FORMULATION

The task of predicting future vehicle traffic can be formulated as follows. Let $s_1, s_2, s_3, \dots, s_m$ represent m observation points or VDS within a region R . Traffic conditions are characterized by four parameters: timestamp (T), flow (F), speed (S), and occupancy rate (O), which are used to assess spatiotemporal dependencies. $F^{s_1}, S^{s_1}, O^{s_1} \dots F^{s_m}, S^{s_m}, O^{s_m}$ represents the flow, speed, and occupancy rate at VDS segments $s_1, s_2, s_3, \dots, s_m$, respectively. Let $t_1, t_2, t_3, \dots, t_n$ represent the time steps at which the traffic parameters are measured at the VDS, with measurements taken at regular intervals (e.g., one hour). The traffic flow, speed, and occupancy rate at VDS s_1 at time t_1 are represented by $F_{t_1}^{s_1}, S_{t_1}^{s_1}, O_{t_1}^{s_1}$ respectively. The goal is to predict the future traffic speed at time t_{h+1} , given historical data on traffic flow, speed, and occupancy rate from the VDS for h past time steps ($t_1, t_2, \dots, t_h$). In this work, $h = 24$, meaning that data from the past 24 hours are used to predict traffic speed for the next hour.

RNN variants are highly effective for processing time series data, as they excel at capturing temporal dependencies. Additionally, these models can be adapted to understand spatial relationships, making them suitable for spatiotemporal sequence analysis. To effectively leverage RNN variants for capturing both temporal and spatial dependencies, it is essential to structure the time series data as spatial sequences. This involves organizing data from multiple observation points (e.g., roads, stations, VDS) in a sequence that reflects their physical arrangement. Each data point in the sequence represents traffic parameters (flow, speed, occupancy rate)

at a specific location and time along the highway. Although limited attention has been given to this structured approach in the literature, it enables a comprehensive analysis of traffic dynamics by fully exploiting spatiotemporal relationships. The structured approach can be represented as a spatiotemporal matrix $X_{T_{t_1:t_n}}^R$, which includes the traffic parameters from VDS 1 to m in a region R and reflects the physical arrangement of the VDS (i.e., s_1 follows s_2 and so on). By using RNN variants on the spatiotemporal matrix, it is possible to capture both temporal patterns (e.g., hourly traffic fluctuations) and spatial dependencies (e.g., inter-VDS interactions), thereby improving the accuracy of traffic speed prediction.

$$X_{T_{t_1:t_n}}^R = \begin{bmatrix} f_{t_1}^{s_1} & S_{t_1}^{s_1} & O_{t_1}^{s_1} & f_{t_1}^{s_2} & S_{t_1}^{s_2} & O_{t_1}^{s_2} & \dots & f_{t_1}^{s_m} & S_{t_1}^{s_m} & O_{t_1}^{s_m} \\ f_{t_2}^{s_1} & S_{t_2}^{s_1} & O_{t_2}^{s_1} & f_{t_2}^{s_2} & S_{t_2}^{s_2} & O_{t_2}^{s_2} & \dots & f_{t_2}^{s_m} & S_{t_2}^{s_m} & O_{t_2}^{s_m} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ f_{t_h}^{s_1} & S_{t_h}^{s_1} & O_{t_h}^{s_1} & f_{t_h}^{s_2} & S_{t_h}^{s_2} & O_{t_h}^{s_2} & \dots & f_{t_h}^{s_m} & S_{t_h}^{s_m} & O_{t_h}^{s_m} \\ f_{t_{h+1}}^{s_1} & S_{t_{h+1}}^{s_1} & O_{t_{h+1}}^{s_1} & f_{t_{h+1}}^{s_2} & S_{t_{h+1}}^{s_2} & O_{t_{h+1}}^{s_2} & \dots & f_{t_{h+1}}^{s_m} & S_{t_{h+1}}^{s_m} & O_{t_{h+1}}^{s_m} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ f_{t_n}^{s_1} & S_{t_n}^{s_1} & O_{t_n}^{s_1} & f_{t_n}^{s_2} & S_{t_n}^{s_2} & O_{t_n}^{s_2} & \dots & f_{t_n}^{s_m} & S_{t_n}^{s_m} & O_{t_n}^{s_m} \end{bmatrix}$$

Before presenting our traffic model, let us define $H_{T_{t_1:t_h}}^R$, which represents the historical traffic flow, speed, and occupancy rate in a region R over the time interval $T[t_1:t_h]$. This interval corresponds to a fixed-size window of past observations that is used to predict future traffic values.

$$H_{T_{t_1:t_h}}^R = \begin{bmatrix} f_{t_1}^{s_1} & S_{t_1}^{s_1} & O_{t_1}^{s_1} & f_{t_1}^{s_2} & S_{t_1}^{s_2} & O_{t_1}^{s_2} & \dots & f_{t_1}^{s_m} & S_{t_1}^{s_m} & O_{t_1}^{s_m} \\ f_{t_2}^{s_1} & S_{t_2}^{s_1} & O_{t_2}^{s_1} & f_{t_2}^{s_2} & S_{t_2}^{s_2} & O_{t_2}^{s_2} & \dots & f_{t_2}^{s_m} & S_{t_2}^{s_m} & O_{t_2}^{s_m} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ f_{t_h}^{s_1} & S_{t_h}^{s_1} & O_{t_h}^{s_1} & f_{t_h}^{s_2} & S_{t_h}^{s_2} & O_{t_h}^{s_2} & \dots & f_{t_h}^{s_m} & S_{t_h}^{s_m} & O_{t_h}^{s_m} \end{bmatrix}$$

The methodology section explains the problem of predicting the future traffic speed $S_{t_{h+1}}^{s_i}$ for a segment s_i at the time step t_{h+1} (the next hour).

IV. BACKGROUND

Establishing an efficient speed prediction framework using advanced machine learning and optimization techniques can significantly enhance traffic flow management and mitigate congestion. This section provides the theoretical background of RF, GA, NSGA-II, and RNN.

A. RANDOM FOREST REGRESSION

Random Forest Regression (RFR) is an ensemble learning technique that utilizes multiple regression trees to predict continuous outcomes. Regression tree is organized in a hierarchical structure, resembling a tree with nodes and branches. The nodes represent decision points based on the values of features, while the branches indicate the possible outcomes of those decisions. The predicted values for the target variable are located in the leaf nodes. The tree is constructed using a top-down greedy approach, which

recursively splits the observations at each node until a stopping condition is reached.

The splitting criterion for a regression task in a decision tree is based on the reduction in the Sum of Squared Residuals (SSR). For each feature, various thresholds or potential split points are evaluated, determined as the midpoints between consecutive unique values of that feature. The data is then divided into two subsets for each threshold, and the SSR is computed for each subset. The threshold that results in the smallest total SSR across the two subsets is selected as the optimal split point for that feature. Among the optimal splits identified for each feature, the one with the smallest overall SSR is chosen as the best split for the root node. This process is recursively applied to further split the data at subsequent nodes. The recursion continues until a node meets one of the stopping criteria, such as having fewer than a specified number of observations or reaching a pre-defined maximum tree depth. When a node satisfies the stopping criteria, it becomes a leaf node. In RFR, multiple regression trees are trained on different subsets of the data obtained through bootstrapping. Additionally, at each split within a tree, only a random subset of features is considered for selection. This introduces further randomness and helps reduce overfitting. The control flow process of RFR is illustrated in the left portion of Figure 5.

- 1) Create a dataset containing input features and target variables suitable for the problem.
- 2) Apply normalization or standardization to the dataset to ensure all features are on a similar scale. Then, divide the dataset into training and test sets.
- 3) Generate a bootstrapped dataset of the same size as the training set by randomly selecting samples with replacements from the training set.
- 4) Using the bootstrapped dataset, construct a regression tree. Consider only a random subset of the available features at each split to determine the optimal split.
- 5) Repeat steps 3 and 4 to generate multiple bootstrapped datasets and build a corresponding regression tree. Again, consider only a random subset of features at each split.
- 6) Pass the data through all the regression trees to obtain individual predictions for unseen data points. Average these predictions to compute the final value, as shown in Equation (1).

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N f_i(x) \quad (1)$$

where, $f_i(x)$ is the prediction made by the i^{th} tree for input x , and $\hat{y}$ is the final predicted value.

- 7) Evaluate the model on the test set using metrics such as RMSE, MSE, or R^2 .

B. GENETIC ALGORITHM

The genetic algorithm, based on the principle of “survival of the fittest,” is a heuristic search algorithm that leverages ran-

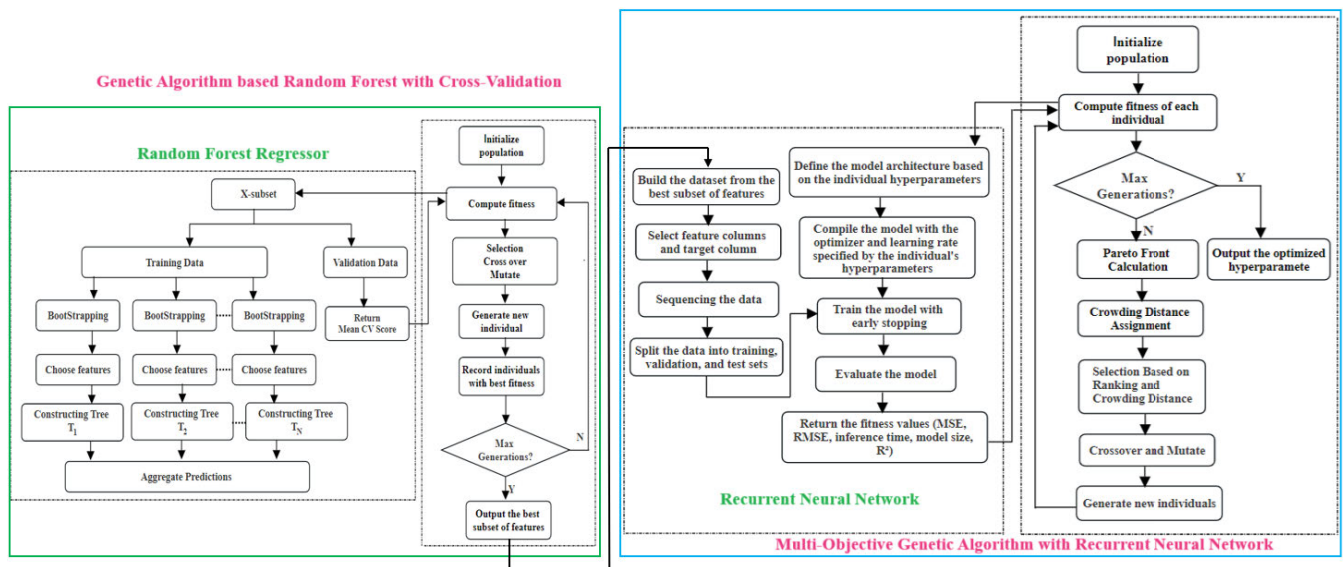


FIGURE 5. Integration of GA with RF-CV and RNN for multi-objective optimization.

domization and evolutionary principles to find approximate solutions quickly and effectively. The control flow process of GA integrated with RF-CV is presented in the left portion of Figure 5. The left section of Figure 5 presents the control flow process of GA integrated with RF-CV. It efficiently explores large subsets of the search space through the following steps:

- 1) Generate an initial population of random individuals, where each individual represents a potential solution.
- 2) Compute the fitness score of each individual in the population to assess their quality or suitability for solving the problem.
- 3) Use a selection operator to choose fitter individuals based on their fitness scores. These selected individuals will serve as parents to produce the next generation.
- 4) Create new individuals (offspring) by performing a crossover operation, where the genetic material of selected parents is combined at specified crossover points to produce offspring.
- 5) Apply a mutation operator to introduce random changes in the offspring's genetic material. This step helps maintain diversity in the population and prevents premature convergence.
- 6) Replace less fit individuals with the new offspring and evaluate their fitness. Repeat steps 3 to 6 until a predefined termination condition is met, such as reaching a specified number of generations.

C. NSGA II

NSGA II is a multi-objective optimization algorithm designed to solve problems with multiple conflicting objectives by extending the principles of GA. Unlike single-objective optimization, NSGA-II provides diverse solutions represented as a Pareto front. The control flow process of

NSGA-II integrated with RNN is illustrated in the right section of Figure 5. The step-by-step procedure of NSGA-II is described as follows:

- 1) A random population of solutions is generated, each representing a combination of decision variables.
- 2) A set of fitness values corresponding to the multiple objectives is computed for each individual in the population. These fitness values form the basis for further sorting and selection procedures.
- 3) The individuals in the population are sorted into different Pareto fronts based on their dominance relationships: Front 1 contains non-dominated solutions (not dominated by any other solution in the population). Front 2 contains solutions dominated by solutions only in Front 1. Front 3 contains solutions dominated by solutions in Front 1 and Front 2. This process continues for subsequent fronts, with each front containing solutions that are progressively less optimal, as they are dominated by solutions in higher-priority fronts.

DOMINATION

Solution a dominates b if a is better or equal in all the objectives than b and is strictly better in at least one objective.

NON-DOMINATION

A solution is not dominated if no solution dominates it in all objectives.

- 4) Calculating the crowding distance for each individual in the same front. Within the same front, the individuals with the highest crowding distance are preferably selected, as they are isolated from the other solutions.

CROWDING DISTANCE

- Sort the individuals based on each objective.
- Assign infinite crowding distance to the boundary individuals.
- Calculate the crowding distance (C_dst) for each individual concerning each objective (Obj) via equation (2).

$$C_dst = \sum \left(\frac{\Delta Obj}{\text{Max Value} - \text{Min Value}} \right) \quad (2)$$

where,

$$\Delta Obj = \text{Next obj value} - \text{Previous obj value}$$

- 5) Selecting the individuals based on the front number (rank) and crowding distance; the number of individuals chosen typically equals the population size.
- 6) Performing cross-over and mutation operations to generate a set of offspring individuals of population size.
- 7) Generating a single pool by merging the parent and offspring populations following the $\mu + \lambda$ strategy, where μ is the size of the parent population and λ is the number of offspring generated.
- 8) Sorting of the merged pool, with non-dominated sorting and crowding distance, to output the sorted fronts with crowding distances for each individual.
- 9) Forming a population for the next generation by selecting individuals beginning with best-numbered fronts to reach population size.
- 10) Repeat steps 2 to 9 for a defined number of generations or stopping criteria that are met.

D. RECURRENT NEURAL NETWORK

RNN is a neural network specifically designed to model sequence learning problems. In such problems, the subsequent output depends on the previous and current inputs at any given time step. RNNs process the input sequentially, one step at a time while maintaining an internal or hidden state that retains information about past inputs. The right section of Figure 5 presents the step-by-step process of an RNN integrated with MOGA (NSGA-II). As depicted in Figure 6, the utilization of recurrent connections between hidden states in an RNN ensures the following:

- 1) The same transfer function is applied at every time step due to parameter sharing, ensuring consistency and reducing the complexity of the model.
- 2) The transfer function incorporates information from both the current input and the previous hidden state, enabling the network to capture temporal dependencies in the data.
- 3) The transfer function can handle sequences with a variable number of inputs, making RNNs versatile for tasks involving sequential data of varying lengths.

The RNN transfer function is formulated using equations (3) and (4).

$$q_{t_h} = f(Ux_{t_h} + Wq_{t_{h-1}} + b) \quad (3)$$

$$y_{t_h} = O(V \cdot q_{t_h} + c) \quad (4)$$

In equation (3), x_{t_h} is the input feature vector in time step t_h with dimensions $\mathbb{R}^z$, q_{t_h} and $q_{t_{h-1}}$ are the current and previous hidden states at time steps t_h and t_{h-1} , respectively, with dimensions $\mathbb{R}^d$. The number of neurons in the hidden layer is d and U is the weight matrix that connects the current input x_{t_h} and the current hidden state with dimensions $\mathbb{R}^{d \times z}$. W is the weight matrix for the recurrent connections between the hidden states with dimensions $\mathbb{R}^{d \times d}$, b represents the bias vector of dimensions $\mathbb{R}^d$, and $f(\cdot)$ applies the nonlinearity of the elements in the form of an activation function. Equation (4) is used for regression tasks, where, y_{t_h} represents the predicted output at the time step t_h , V is the output weight matrix with dimensions $\mathbb{R}^{k \times d}$, where k is the number of predicted values, c is the bias vector with dimensions $\mathbb{R}^k$ and $O(\cdot)$ is the activation function.

V. METHODOLOGY

This research examines three key traffic parameters: flow, speed, and occupancy rate. Together, these parameters provide a comprehensive understanding of traffic conditions. Both flow and occupancy rates are essential for analyzing traffic dynamics. In particular, the occupancy rate is significant because it reflects potential speed reductions due to its relationship with vehicle density and road space utilization. Our focus is on predicting the short-term traffic speed of a specific road segment by capturing subtle and significant variations in speed that are influenced by traffic flow, occupancy rate, and speed fluctuations. These variations are analyzed in the hourly patterns of the target segment alongside its neighbouring segments. Identifying the most relevant features that are spatiotemporally correlated with the target variable is crucial for achieving accurate predictions.

To achieve this, our work employs feature selection using a Genetic Algorithm (GA) combined with Random Forest Cross-Validation (RF-CV). The GA promotes diversity in feature subsets and helps reduce the risk of overfitting by introducing randomness through crossover and mutation. Meanwhile, RF-CV is a robust model that can handle non-linear relationships and is well-suited for capturing the intricate dependencies in traffic data. We also perform hyperparameter optimization of the model using the NSGA-II multi-objective optimization algorithm. This approach ensures a balanced trade-off among model complexity, stability, and predictive accuracy, ultimately leading to reliable and precise short-term traffic speed predictions.

A. RELEVANT FEATURE SELECTION

The left side of Figure 5 depicts the flowchart of the proposed feature selection approach. Algorithm 1 outlines the genetic algorithm integrated with RF-CV for feature

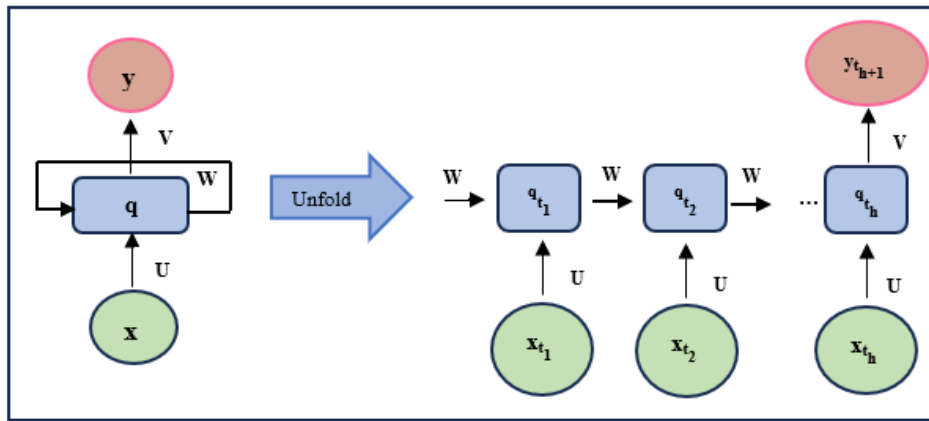


FIGURE 6. RNN structure.

TABLE 5. GA and RF parameter values.

Parameter	Value	Parameter	Value
POP_SIZE	50	n_estimators	100
P_CROSSOVER	0.9	k	5
P_MUTATION	0.1	Criterion	MSE
MAX_GENE	25 \50		
HOF_SIZE	5		
CROSSOVER	2PX		

selection, which is implemented using the DEAP framework. This algorithm requires a dataset that includes predictors X and a target variable y . The predictors consist of flow, speed, and occupancy rate data collected from five VDS segments. The primary objective is to predict the speed at the third VDS. The initial parameter settings for the GA-based RF-CV approach are summarized in Table 5. Cross-validation is utilized to divide the dataset into a predefined number of folds (k). The model is trained on $k-1$ subsets and validated on the remaining subset. This methodology ensures a robust model evaluation and minimizes the risk of overfitting.

Once the required parameters are initialized, an RFR with cross-validation is defined as the evaluation model, using a specified number of estimators ($n_estimators$) with the function `RandomForestRegressor()`. The function `convert_to_boolean(individual)` translates a genetic encoding (0s and 1s) into a feature selection mask. The regressor then evaluates each individual generated by the genetic algorithm and computes the mean cross-validation score on the validation set using the function `cross_val_score()`. The population is initialized using `initialize_population( $P_s$ )`, and the best-performing individuals are stored in the Hall of Fame (HOF) using `initialize_hall_of_fame( $HOF_s$ )`. The HOF is a mechanism that tracks the best-performing individuals across generations. It ensures that top solutions are preserved, even if the population evolves differently in later generations.

Algorithm 1 Feature Selection With Genetic Algorithm

- 1: **Input:** Traffic Dataset, GA parameters: population size P_s , number of generations N_{gen} , mutation probability M_{pb} , crossover probability C_{pb} , hall of fame size HOF_s , number of folds k and random seed R_{seed}
- 2: **Output:** Relevant feature subset
- 3: **Load the Dataset**
- 4: $X \leftarrow \text{dataset}[\text{Predictors}]$
- 5: $y \leftarrow \text{dataset}[\text{target}]$
- 6: **Initialize GA Parameters:**
 $P_s, N_{gen}, M_{pb}, C_{pb}, HOF_s, k, R_{seed}$
- 7: **Define the Random Forest Regressor**
- 8: $\text{reg} \leftarrow \text{RandomForestRegressor}(n_estimators)$
- 9: **Define Fitness Function**
- 10: $\text{mask} \leftarrow \text{convert_to_boolean}(\text{individual})$
- 11: **if** $\text{sum}(\text{mask}) == 0$ **then**
- 12: **return** 0
- 13: **else**
- 14: $X_{\text{subset}} \leftarrow \text{select_features}(X, \text{mask})$
- 15: $\text{cv_scores} \leftarrow \text{cross_val_score}(\text{reg}, X_{\text{subset}}, y, k)$
- 16: **return** $\text{mean}(\text{cv_scores})$.
- 17: **Initialize Population and GA Tools**
- 18: $P \leftarrow \text{initialize_population}(P_s)$
- 19: $\text{hof} \leftarrow \text{initialize_hall_of_fame}(HOF_s)$
- 20: $\text{stats} \leftarrow \text{initialize_statistics}()$
- 21: **Run Genetic Algorithm**
- 22: $P, \text{logbook} \leftarrow \text{eaSimple}(P, N_{gen}, C_{pb}, M_{pb}, \text{stats}, \text{hof})$
- 23: **Extract and Save Results**
- 24: $\text{best_ind} \leftarrow \text{extract_best_individual}(\text{hof})$
- 25: $\text{best_features} \leftarrow \text{convert_to_boolean}(\text{best_ind})$
- 26: $\text{selected_features} \leftarrow \text{apply_mask}(X, \text{best_features})$
- 27: $\text{selected_features}[\text{target}] \leftarrow y$

The function `eaSimple()` executes the evolutionary process, applying selection, crossover, and mutation over multiple generations to evolve feature subsets. Finally, the best individual is extracted and converted into a feature mask, which is

then applied to the dataset to obtain the final selected feature subset using the functions `extract_best_individual(hof)` and `apply_mask(X, best_features)`.

Unlike simple correlation metrics that only capture linear relationships, RF-CV can effectively capture the nonlinear relationships between the input features and the target variable. Identifying these nonlinear relationships is essential for uncovering the most critical spatiotemporal dependencies contributing to the prediction. The aggregated cross-validation score is an ideal criterion for feature selection in complex datasets, as it ensures the model performs consistently well across different folds, thereby reducing the risk of overfitting. Using the AMSE score in the objective function is crucial because it ensures that the selected feature subset generalizes well to unseen data. The objective function evaluates the AMSE for each individual (feature subset) the genetic algorithm generates. The selection operation then chooses the best-performing feature subsets based on the lowest AMSE. The feature selection process involves the following key steps:

- 1) Create initial populations using a random seed.
- 2) Training the RF-CV on each feature subset X_{subset} , which is the feature subset matrix obtained from the original set, where $X_{\text{subset}} \subseteq X_{T_1:T_n}^R$.
- 3) Compute and return the mean of the K-fold cross-validation score on the validation set as the fitness of the individual, computed as $\text{CV}(\text{RF}(X_{\text{subset_val}}, y_{\text{val}}))$.
- 4) Selecting individuals for reproduction based on their fitness score.
- 5) Applying crossover and mutation operations to generate new offspring.
- 6) Evaluating the new population.
- 7) Repeating the cycle for a defined number of generations.

Each generation stores the best-performing individuals in a Hall of Fame. After a predefined number of generations, the genetic algorithm determines the optimal feature subset using Equation (5).

$$X_{\text{selected}} = \underset{X_{\text{subset}}}{\text{FitnessMax}} \left(\text{GA} \left(\text{CV} \left(\text{RF} \left(X_{\text{subset_val}}, y_{\text{val}} \right) \right) \right) \right) \quad (5)$$

where $\text{FitnessMax}_{X_{\text{subset}}}$ represents the process of finding the best subset of features (X_{selected}) that maximizes the fitness function based on the AMSE score returned by the RF-CV on the validation set. $\text{GA}(\dots)$ indicates that a genetic algorithm is used to explore through different feature subsets to optimize this process.

The computational complexity of the feature selection algorithm is given by $O(N_{\text{gen}} \cdot P_s \cdot k \cdot t \cdot d \log(d))$, where: N_{gen} : Number of generations in the genetic algorithm, P_s : Population size used in the optimization process, k : Number of cross-validation folds, t : Number of trees in the random forest model, and d : Dimensionality of the input data.

B. HYPERPARAMETER OPTIMIZATION

The right side of Figure 5 illustrates the flowchart of hyperparameter optimization using an NSGA-II framework integrated with an RNN model. Algorithms 2, 3, and 4 detail the entire process of hyperparameter optimization. A systematic search of RNN hyperparameters is performed to balance the model's performance and stability. The hyperparameters optimized include the number of RNN layers (n_{rl}), number of neurons (n_n), learning rate (lr), batch size (bs), dropout rate (dr), activation function (af), optimizer (opt_alg), weight initialization (wi), sequence length (seq_len), and early stopping patience (esp). Among these, lr , wi , and opt_alg contribute to stable training, while n_{rl} , n_n , and af are crucial for learning complex patterns.

Algorithm 2 NSGA-II Based Hyperparameter Optimization

- 1: **Input:** Traffic dataset, GA parameters: population size P_s , number of generations N_{gen} , mutation probability M_{pb} , crossover probability C_{pb}
- 2: **Output:** Optimal hyperparameters
- 3: **Load the dataset**
- 4: $X \leftarrow \text{dataset}[X_{\text{selected}}]$
- 5: $y \leftarrow \text{dataset}[\text{target}]$
- 6: **Initialize GA parameters:** $P_s, N_{\text{gen}}, M_{pb}, C_{pb}$
- 7: $\text{population} \leftarrow \text{initializePopulation}(P_s)$
- 8: **for** generation = 1 to N_{gen} **do**
- 9: **for** each individual in population **do**
- 10: $\text{fitness_metrics} \leftarrow \text{evaluate}(\text{individual})$
- 11: Store fitness_metrics for further processing
- 12: **end for**
- 13: Apply NSGA-II operations: Pareto front calculation, Crowding Distance, Selection, Crossover, Mutation, Merging and Sorting
- 14: Generate a new population
- 15: **end for**
- 16: $\text{best_individual} \leftarrow \text{selBest}(\text{population}, 1)$
- 17: **return** $\text{best_individual}[0].\text{fitness.values}$

The multi-objective optimization using NSGA-II aims to minimize MSE, RMSE, inference times, and model size while maximizing R^2 . By simultaneously optimizing these objectives, NSGA-II produces a Pareto front of non-dominated solutions, offering a set of optimal trade-offs. Before initiating the multi-objective optimization, the dataset for training the RNN model is prepared based on the relevant feature subset identified using Algorithm 1. The predictors include flow, speed of VDS 1, speed of VDS 2, and speed of VDS 3, while the target variable is the speed of VDS 3. After sequencing and splitting the data, the RNN model is trained using the individuals generated by NSGA-II, enabling the identification of the best hyperparameters. The steps in the optimization process are as follows:

- 1) The initial population, consisting of 50 individuals, is generated, each representing a specific configuration

Algorithm 3 RNN Evaluation Function

```

1: procedure Evaluate(individual)
2:   Extract hyperparameters from individual:
3:   (nr, nn, af, dr, seqlen, bs, lr, optalg, wi, esp)
4:   Generate input sequences
5:   Partition data into training, validation, and test sets
6:   Build RNN model
7:   Select optimizer
8:   Compile model using MSE loss function
9:   Train model with early stopping
10:  Predict on test data
11:  if any predicted value is invalid then
12:    return NaN
13:  end if
14:  Inverse scale predictions
15:  Inverse scale actual values
16:  Compute MSE (mse) and  $R^2$  score ( $r^2$ )
17:  Measure inference time (inftm)
18:  Compute model size
19:  return (mse, rmse, inftm, model size, r2)
20: end procedure

```

Algorithm 4 Generating Initial Population for NSGA-II

```

1: procedure InitializePopulation(num_individuals)
2:   Initialize an empty list for the population
3:   for  $i = 1$  to num_individuals do
4:     Create an empty dictionary for hyperparameters
5:     Assign random values to hyperparameters:
6:     ( $n_{r1}, n_n, lr, bs, dr, af, opt, wi, seq\_len,$ 
7:      $esp$ )
7:     Append the individual to the population list
8:   end for
9:   return population
10: end procedure

```

of the RNN model's hyperparameters based on Algorithm 4. The initial settings for the RNN hyperparameters are detailed in Table 6.

- 2) Computing each individual's fitness, where the fitness function returns the MSE, RMSE, inference time, model size, R^2 of the RNN model compiled, trained and evaluated on the individual configurations.
- 3) Classifying the individuals into different fronts using non-dominated sorting, where the solutions in the front rank the lowest.
- 4) Calculating crowding distance for each individual within the front, where a higher crowding distance implies better fitness in the same front.
- 5) Selecting individuals based on ranking and crowding distance, which become parents for further operations and the number of chosen individuals is based on the population size.
- 6) Performing crossover operation using uniform crossover operator with 0.5 as the crossover probability

TABLE 6. RNN hyperparameters.

Hyperparameter	Initial Setting	Optimized Value
RNN layers	1-3	2
No. of neurons	32-128	84
Learning rate	0.0001-0.01	0.0051735
Batch size	16\32\64	16
Dropout rate	0.1-0.5	0.333004
Activation function	relu\tanh\sigmoid	relu
Optimizer	adam\SGD\RMSprop	adam
Weight initialization	glorot_uniform\he_normal	he_normal
Sequence length	12-24	13
Early stopping patience	3-10	9

so that each hyperparameter gets a 50% chance of being swapped between parents.

- 7) Performing mutation operation, with the mutation probability of 0.1 and custom mutate function. It applies a small random change within $\pm 10\%$ of the current value for numeric and integer parameters while clamping new values within bounds. Also, the rounding of integers is applied to restore its type. From the predefined choices, new random values are replaced for categorical parameters.
- 8) Combining parent and offspring populations to merge into a single pool followed by non-dominated sorting and crowding distance calculation to rank the individuals.
- 9) Generating next-generation population from the sorted merged pool, starting from front1.
- 10) Repeat steps 2 to 9 until the defined number of generations is reached to optimize the hyperparameters of the RNN model. The hyperparameter values optimized using the NSGA-II algorithm are presented in Table 6.

The computational complexity of hyperparameter optimization is $O(N'_{gen} \cdot p'_s \cdot e \cdot d \cdot seq_len \cdot r^2)$, where: N'_{gen} : Number of generations in the NSGA-II algorithm, p'_s : Population size in NSGA-II, e : Number of epochs used during RNN training, d : Dimensionality of the input data, seq_len : Length of the input sequence, and r : Weight matrices of size $r \times r$. The training complexity of the RNN model is $O(e \cdot d \cdot seq_len \cdot r^2)$, as it depends on the number of epochs, the input dimensionality, the sequence length, and the size of the weight matrices.

C. RNN TRANSFER FUNCTION AND SIGNIFICANCE

This subsection describes the RNN transfer function, its significance in vehicle traffic prediction, and a practical application scenario. Historical spatiotemporal dependencies in traffic flow, speed, and occupancy significantly influence future traffic conditions. This research proposes a MOGA-based RNN framework to capture these dependencies effectively. While RNNs are inherently designed to model temporal

sequences, this work extends their capability to account for spatial dependencies by integrating relationships between upstream and downstream VDS segments. Relevant spatial features are incorporated into the temporal input vector, selected using a GA-based RF-CV approach. These selected relevant features include flow, speed, and occupancy rates from adjacent VDS segments (VDS1 to VDS5), which collectively affect the spatiotemporal dynamics at a specific segment, such as VDS3.

By sequentially processing relevant features, the RNN learns two key aspects:

- **Spatial Relationships:** The RNN captures how upstream and downstream VDS influence the traffic conditions at VDS3 by learning the interactions and dependencies among these spatially related nodes.
- **Temporal Dependencies:** The RNN model learns how past traffic conditions influence future outcomes by evolving its hidden state q_{t_h} to effectively retain and process information from earlier time steps.

In a spatiotemporal context, the transfer function of an RNN mathematically presented in equation (3) can be expressed as follows:

x_{t_h} : Input vector at time step t_h contains spatial features, such as flow, speed, and occupancy, for adjacent VDS: VDS1, VDS2, VDS3, VDS4, and VDS5.

q_{t_h} : Represents the hidden state at time step t_h , encoding the temporal and spatial context from the current input.

$q_{t_{h-1}}$: The hidden state from the previous time step, which provides temporal continuity by propagating learned information across time steps.

U : The weight matrix governing input-to-hidden connections.

W : The weight matrix used for hidden-to-hidden (recurrent) connections, capturing dependencies between consecutive hidden states.

b : Denotes the bias vector which adjusts the weighted sum before applying the activation function.

$f(\cdot)$: The activation function that introduces non-linearity to the hidden layer, enabling the model to capture complex relationships in the data. just tell me what is output of this latex code.

The following are the key points that emphasize the significance of the RNN transfer function:

- The hidden state q_{t_h} acts as a memory unit, retaining temporal dependencies across time steps in the sequence.
- By combining q_{t_h} with $q_{t_{h-1}}$, the RNN effectively captures the spatiotemporal interactions that are crucial for predicting traffic dynamics at VDS3.

For speed prediction we have used equation (4) and expressed it as follows:

$y_{t_{h+1}}$: The predicted speed of VDS3 at time step t_{h+1}

V : The weight matrix that connects the hidden state to the output layer.

c : The bias vector for the output layer.

TABLE 7. Initial hyperparameter configuration.

Hyperparameter	Configuration
Layers	3
Activation function	ReLU
Optimizer	ADAM
Loss Function	MSE
Batch Size	32
Normalization	MinMaxScaler
Epochs	100

$O(\cdot)$: The output activation function applied to the sum of the weighted hidden state and bias.

The following point highlights the significance of the output function in an RNN.

- The prediction $y_{t_{h+1}}$ depends on the hidden state q_{t_h} , which encodes the combined spatiotemporal context of both upstream and downstream segments over the given sequence length.

Practical Scenario: Consider a scenario with a sequence length of 24, where the GA-based relevant features include the following:

- f^{s_1} : Traffic flow at VDS1.
- S^{s_1} , S^{s_2} , S^{s_3} : Traffic speeds at VDS segments 1, 2, and 3, respectively.

The RNN processes data in the following manner:

- 1) **Input Vector:** Each input vector x_{t_h} represents the relevant spatial features at time step t_h . For $h=1$, $x_{t_1} = [f_{t_1}^{s_1}, S_{t_1}^{s_1}, S_{t_1}^{s_2}, S_{t_1}^{s_3}]$ here, $f_{t_1}^{s_1}$ denotes the flow value of segment s_1 at time t_1 , and $S_{t_1}^{s_1}$, $S_{t_1}^{s_2}$, and $S_{t_1}^{s_3}$ represent additional correlated speed values for other selected locations.
- 2) **Hidden State Update:** At each time step, the hidden state q_{t_1} is updated using the following equation: $q_{t_1} = f(U \cdot x_{t_1} + W \cdot q_{t_0} + b)$ here, q_{t_1} combines the current input x_{t_1} with the temporal context provided by the previous hidden state q_{t_0} .
- 3) **Prediction:** After processing the full sequence $(x_{t_1}, x_{t_2}, \dots, x_{t_{24}})$, the RNN predicts the speed at $y_{t_{25}}$ using the final hidden state $q_{t_{24}}$:

$$y_{t_{25}} = O(V \cdot q_{t_{24}} + c)$$

The process shifts forward to the next sequence $(x_{t_2}, x_{t_3}, \dots, x_{t_{25}})$ to predict $y_{t_{26}}$, and so on. This iterative mechanism allows the RNN to model sequential spatiotemporal dependencies effectively. Figure 6 illustrates this scenario, showcasing the RNN's sequential processing and prediction mechanism.

VI. EXPERIMENTAL RESULTS AND ANALYSIS

A. EXPERIMENTAL SETUP

The experiments were conducted using Google Colab, a cloud-based platform that offers access to powerful hardware accelerators such as GPUs and TPUs. All models

were developed and trained using Python 3.8 in the Google Colab environment. The key libraries include TensorFlow 2.6, Keras 2.6, Scikit-learn 0.24.2, and OpenCV 4.5.3. TensorFlow is an open-source framework for developing machine learning models. Keras is a high-level neural network API that simplifies deep learning models' design, debugging, evaluation, and visualization. Scikit-learn is a machine learning library for data pre-processing, model evaluation, and dataset splitting. Seaborn and Matplotlib were utilized for data visualization and plotting.

The traffic dataset used for the experimentation and evaluation of our proposed methodology was collected from the PeMS [50], which is managed by the California Department of Transportation. PeMS gathers real-time traffic data from over 39,000 individual detectors throughout the freeway systems in all major metropolitan areas of California. The primary data source for PeMS, known as VDS, provides real-time information on traffic flow, speed, and occupancy at 30-second daily intervals.

Our study gathered data from five VDSs on freeway SR 99 in District 10, Merced County, California. Figure 7 illustrates the layout of these VDSs, detailing:

- 1) The unique IDs of the VDS are:
 - VDS 10105810 - VDS 1
 - VDS 10105510 - VDS 2
 - VDS 10105310 - VDS 3
 - VDS 10105110 - VDS 4
 - VDS 10108710 - VDS 5
- 2) The direction of traffic flow, which moves downstream from higher to lower mileposts (represented from left to right in the figure).
- 3) The locations of onramps (entry points to the freeway) and offramps (exit points from the freeway).

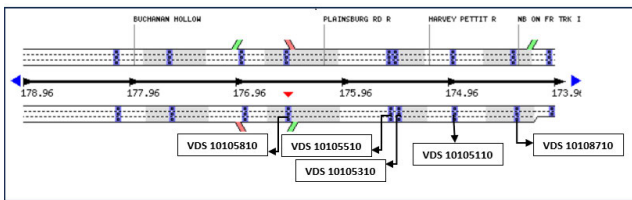


FIGURE 7. PeMS VDS layout.

Additionally, the layout includes milepost markers such as 178.96, 177.96, 176.96, 175.96, 174.96, and 173.9, as well as landmarks like “Buchanan Hollow,” “Plainsburg Rd,” “Harvey Pettit,” and “NB ON FR TRK 1” to provide context for the monitored freeway segments and the traffic data collection process. Although the figure does not explicitly show them, each VDS is implied to monitor a specific freeway segment. This segment is defined by the distance to the nearest upstream and downstream VDS, ensuring comprehensive traffic monitoring. The dataset used for analysis consists of hourly traffic flow, speed, and occupancy rate data collected from each VDS, spanning from 12-01-

2021 12:00:00 to 02-28-2024 23:00, resulting in 19,678 instances with 15 features. These features include traffic parameters (flow, speed, and occupancy) recorded at the five VDS stations.

B. EXPERIMENTAL RESULTS

Our feature selection approach utilizes a genetic algorithm-based random forest with cross-validation to identify the most relevant features from the dataset. This method effectively filters out redundant and irrelevant features, yielding the best subset that captures the complex spatiotemporal relationships with the target variable. The fitness function is designed based on the mean cross-validation score to select the optimal feature subset, evaluating their combined contribution to the model's predictive performance. Table 5 outlines the parameter settings used in our feature selection method.

For twenty-five generations, the relevant features extracted for speed predictions at VDS3 are f^{s1} , S^{s1} , and S^{s3} , whereas, for 50 generations, only S^{s3} is selected. In Figure 8, we have compared the basic and state-of-the-art foundational models such as RNN, GRU, LSTM, Bi-LSTM, LSTM-AM, Conv-LSTM, CNN-LSTM and CNN-GRU based on various performance metrics: MSE, RMSE, MAE, R^2 , training, testing and validation time, model size, and the number of parameters to predict S^{s3} . These predictions were made using all the input features f^{s1} , S^{s1} , $O^{s1} \dots f^{s5}$, S^{s5} , O^{s5} , and the GA-based relevant features. The hyperparameter settings used for these predictions are presented in Table 7.

In Figures 8(a), 8(b), and 8(c), it is evident that models utilizing GA-based relevant feature subsets for generations 25 and 50 achieve lower MSE, RMSE, and MAE compared to models using all input features. The CNN-LSTM model has the lowest MSE of 3.58, followed by LSTM-AM and RNN, with MSE values of 3.62 and 3.69, respectively. Furthermore, Figure 8 (d) shows improved R^2 values when using GA-based relevant features. The CNN-LSTM model achieves the highest R^2 of 0.625, followed by RNN and ConvLSTM with R^2 values of 0.614 and 0.606, respectively. The above observations indicate that GA-based feature selection enhances model prediction performance. The models take longer to converge when using relevant features, as shown in Figures 8(e) and 8(f). However, relevant feature selection reduces test times, as Figure 8(g) depicts. The RNN achieves the shortest test time of 0.46 seconds for predicting 2951 samples, followed by ConvLSTM at 0.64 seconds when using relevant features. In addition, relevant feature selection reduces model complexity regarding memory usage and the number of model parameters, as shown in Figures 8(h) and 8(i). The RNN model has the fewest parameters, totalling 8,577, with a model size of 143,632 bytes. However, LSTM-AM has the highest number of parameters, totalling 115,841, with a model size of 1,432,608 bytes when using relevant features. Therefore, GA-based feature selection for 25 generations and a population size of 50 has effectively reduced loss, test time, model size, and number of parameters while improving model accuracy.

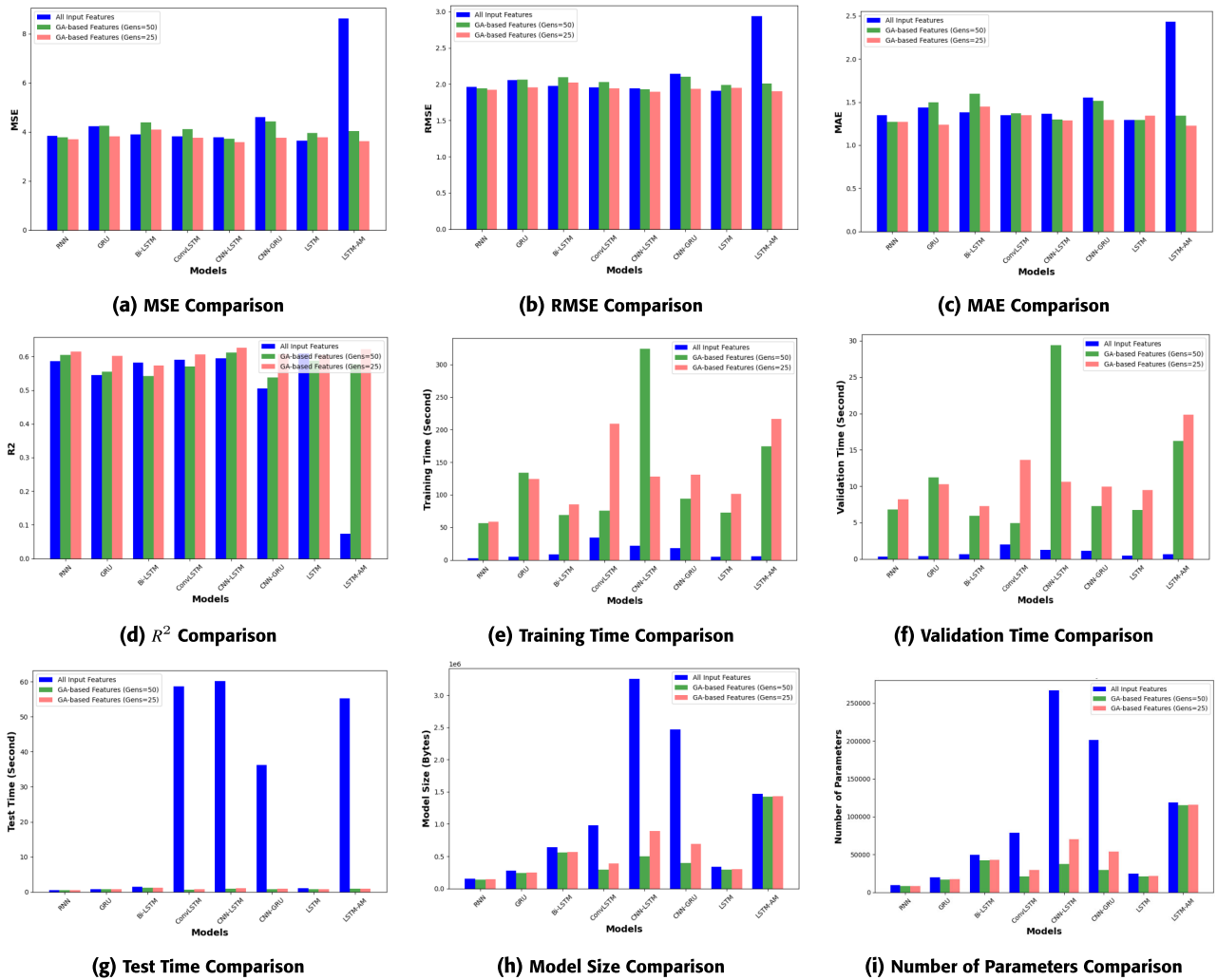


FIGURE 8. Performance comparison of various deep learning models using different input feature sets.

TABLE 8. Analysis of training MSE (T-MSE), Validation MSE (V-MSE), and test MSE (Ts-MSE) for S^3 predictions.

	All Features			GA based Features (Gen-25)			GA based Features (Gen-50)		
	$T - MSE$	$V - MSE$	$Ts - MSE$	$T - MSE$	$V - MSE$	$Ts - MSE$	$T - MSE$	$V - MSE$	$Ts - MSE$
RNN	0.002048	0.001912	3.846961	0.002419	0.001845	3.693973768	0.001701	0.001794	3.781392425
GRU	0.002409	0.001911	4.229474	0.002594	0.001778	3.816338648	0.002649	0.001961	4.257752529
Bi-GRU	0.002601	0.001911	3.891252	0.002423	0.002076	4.086206265	0.00374	0.002275	4.38584285
ConvLSTM	0.00367	0.0022	3.817037	0.003099	0.002172	3.76502564	0.003893	0.002377	4.115694396
CNN-LSTM	0.002754	0.002037	3.77345	0.00287	0.002089	3.583926316	0.003739	0.002068	3.712771807
CNN-GRU	0.002982	0.001972	4.602342	0.002528	0.002091	3.754261999	0.003546	0.001901	4.419183469
LSTM	0.002341	0.001944	3.641522	0.003165	0.002073	3.788433184	0.003368	0.00241	3.947938218
LSTM-AM	0.003855	0.003183	8.620354	0.003702	0.003073	3.620762424	0.00375	0.003187	4.03935378

C. INFERENCE DRAWN

Based on the metrics presented in Figure 8, it can be concluded that the RNN is the least complex model among the evaluated options. While the other models show only marginal improvements in R^2 and RMSE compared to the

RNN, the RNN provides the best balance between test time, number of parameters, and prediction accuracy. This analysis supports the choice of the RNN model for further enhancements in prediction. Although selecting relevant features has improved model accuracy and reduced inference

time, the mean squared error (MSE) of the test set remains significantly higher than that of the training and validation sets, as indicated in Table 8. This discrepancy suggests overfitting, meaning the model has learned noise or spurious patterns rather than the true relationships between the input features and the target variable. To address this issue, hyperparameter tuning can be employed to improve the model's generalization performance on unseen data.

D. COMPARISON STUDY

Our approach to hyperparameter optimization employs the multi-objective genetic algorithm NSGA-II, as elaborated in Section V. This multi-objective framework is instrumental in identifying the optimal combination of hyperparameters for the RNN model. Table 6 delineates the range of hyperparameters that define the search space, alongside the best subset selected through this optimization process. Utilizing the hyperparameters obtained from NSGA-II, in conjunction with L2 regularization, we proceeded to train the RNN model.

In the pre-optimization phase, the speed prediction results at VDS3 are illustrated in Table 8. The lowest test MSE values recorded were 3.58 for the CNN-LSTM model, 3.62 for the LSTM-AM model, and 3.69 for the RNN model. As presented in Table 3, the highest R^2 values achieved were 0.608 for the LSTM, 0.589 for the ConvLSTM model, and 0.581 for the RNN model. Subsequent to the hyperparameter optimization for the RNN model, its performance was compared with that of closely related models developed by Wanneng S. et al. (Bi-GRU model) and Changxi M. et al. (CNN-GRU model), as presented in Table 9 and Figure 9. To ensure an equitable comparison, the optimized hyperparameters from the RNN, as well as the selected features derived from the GA-based RF-CV approach, were applied to both the Bi-GRU and CNN-GRU models. Table 9 summarizes the performance of these models with regard to both regression accuracy and computational efficiency. Table 8 indicates that the test MSE values prior to feature selection and hyperparameter optimization were 3.84 for the RNN model, 3.89 for the Bi-GRU model, and 4.60 for the CNN-GRU model. Following the selected relevant features and optimized hyperparameters, the test MSE values exhibited significant reductions to 0.007865, 0.015554, and 0.142477 for the RNN, Bi-GRU, and CNN-GRU models, respectively, as shown in Table 9.

In parallel, the training durations before feature selection and hyperparameter optimization were 2.60 seconds, 8.52 seconds, and 18.13 seconds for the RNN, Bi-GRU, and CNN-GRU models, respectively. After implementing the selected relevant features and optimized hyperparameters, the training times escalated to 81.64 seconds for the RNN, 67.50 seconds for the Bi-GRU, and 21.20 seconds for the CNN-GRU model. This increase in training duration suggests that the models are engaging in the learning of more complex and relevant feature interactions, thereby resulting in a deeper and more effective learning process. Although the optimized features and hyperparameters necessitate

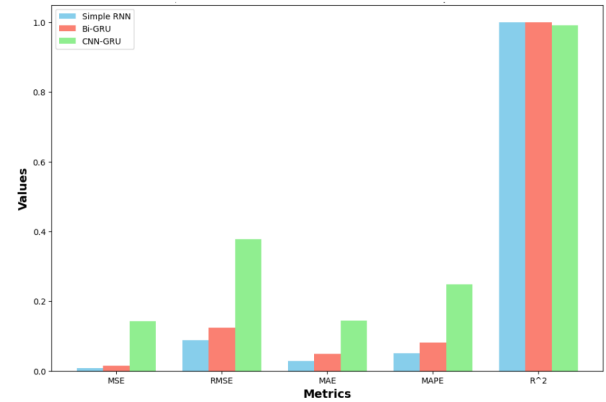


FIGURE 9. Speed prediction evaluation metrics comparison.

additional training time, they substantially enhance predictive performance, as evidenced by the reductions in MSE values. The test durations prior to optimization were recorded as 0.43 seconds for the RNN model, 1.41 seconds for the Bi-GRU model, and 60.13 seconds for the CNN-GRU model. Following optimization, the test durations were decreased to 0.18 seconds, 0.18 seconds, and 0.17 seconds, respectively, indicating an increase in efficiency during inference.

Moreover, Table 9 illustrates that the hyperparameter-optimized RNN model attained the highest R^2 value of 0.9994 and the lowest MSE of 0.00786. This indicates an absolute change of 0.4184 in R^2 and 3.8301 in MSE. Figure 10 compares the actual versus predicted speed values for the first 1,000 test samples across three models: the proposed RNN model, the Bi-GRU model by Wanneng S. et al., and the CNN-GRU model by Changxi M. et al. Each model underwent training utilizing the same selected features and optimized hyperparameters to facilitate a fair comparison. The red dashed line represents the actual speed values, while the predicted values are denoted by solid lines (green, blue, and brown, respectively).

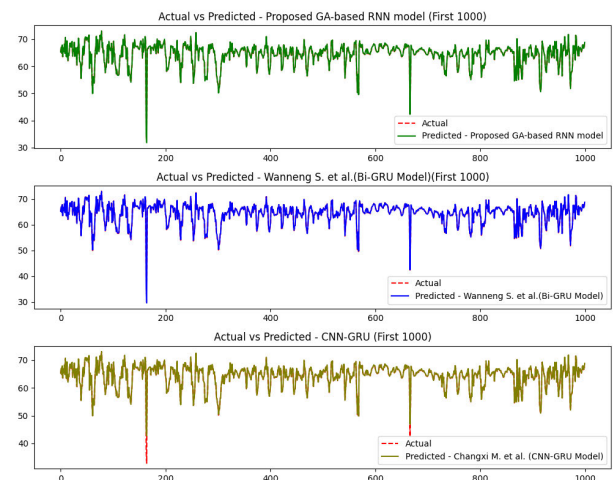


FIGURE 10. Actual vs Predicted speed values.

TABLE 9. Comparative analysis of the proposed models with closely related approaches.

	RNN			Bi-GRU			CNN-GRU		
	<i>Train</i>	<i>Val</i>	<i>Test</i>	<i>Train</i>	<i>Val</i>	<i>Test</i>	<i>Train</i>	<i>Val</i>	<i>Test</i>
MSE	0.052671	0.062093	0.007865	0.026111	0.045444	0.015554	0.017822	0.249164	0.142477
RMSE	0.052671	0.062093	0.007865	0.026111	0.045444	0.015554	0.017822	0.249164	0.142477
MAE	0.028993	0.028993	0.028993	0.048541	0.048541	0.048541	0.145172	0.145172	0.145172
MAPE	0.050961	0.050961	0.050961	0.082052	0.082052	0.082052	0.248800	0.248800	0.248800
R²	0.990190	0.996594	0.999490	0.995137	0.997507	0.998991	0.996680	0.986334	0.990755
Time(sec)	81.64	0.15	0.18	67.50	0.17	0.18	29.20	0.19	0.17

The key observations and insights derived from Figure 10 are as follows:

- 1) **Proposed RNN Model:** The top plot demonstrates a high degree of prediction accuracy, with predicted values closely adhering to actual values. The model effectively captures variations in speed with minimal deviations.
- 2) **Bi-GRU Model by Wanneng S. et al.:** The middle plot reveals that predicted values predominantly align with actual values. However, some discrepancies are observed in areas experiencing rapid speed fluctuations. Nonetheless, the Bi-GRU model outperforms the CNN-GRU model.
- 3) **CNN-GRU Model by Changxi M. et al.:** The bottom plot exhibits noticeable deviations from actual values, particularly in sections where speed experiences significant drops. This suggests that the CNN-GRU model possesses lower predictive accuracy than the GA-based RNN and Bi-GRU models.

These findings correspond with the MSE values in Table 9, where the proposed RNN demonstrates the lowest MSE, succeeded by the Bi-GRU and CNN-GRU models. The results substantiate that feature selection and hyperparameter optimization are critical in enhancing predictive accuracy in vehicle speed forecasting.

E. COMPUTATIONAL EFFICIENCY

As shown in Figures 8 (e), (f), and (g), prior to hyperparameter optimization, the RNN model demonstrated training, validation, and testing times of 58.44 seconds, 8.16 seconds, and 0.46 seconds, respectively, using the GA-based relevant features (Gens=50). Following optimization, the training time presented in Table 9 increased to 81.64 seconds, but the validation and testing times saw significant reductions, dropping to 0.15 seconds and 0.18 seconds. This substantial decrease in validation and testing times enhances the model's efficiency during evaluation and inference, making it more suitable for real-time applications.

Table 10 compares the size and number of parameters for the RNN, Bi-GRU, and CNN-GRU models. The RNN model, with a compact size of 203,118 bytes and only 14,617 parameters, is well-suited for real-time applications requiring

TABLE 10. Model size and parameters comparison.

Model	Model Size	No. of Parameters
RNN	203118	14617
Bi-GRU	746757	59137
CNN-GRU	575857	45281

minimal computational resources. While modern hardware can accommodate larger models, inference time remains critical in latency-sensitive environments, such as autonomous vehicles and intelligent traffic systems. Although the model generates speed predictions hourly, rapid inference is vital for real-time decision-making in areas like congestion control, route optimization, and adaptive traffic signalling. Furthermore, large-scale networks often necessitate multiple predictions simultaneously, making computational efficiency an essential consideration. In edge-based deployments, faster inference reduces energy consumption and ensures smooth operation on embedded devices, enhancing scalability and real-time applicability.

MODEL DEPLOYMENT

The proposed model is designed to operate locally on edge devices rather than in the cloud, as network delays and communication overhead could render real-time predictions impractical. By deploying our proposed model on edge devices such as the NVIDIA Jetson platform, we ensure low-latency responses, along with enhanced privacy, security, availability, and reliability for tasks such as adaptive speed control and traffic optimization. While cloud processing can still be used for managing historical data and updating models, the observed reduction in inference time further confirms that the optimized model improves both accuracy and computational efficiency, making it well-suited for real-time edge AI applications. Therefore, the RNN model, with its relevant features and optimized hyperparameters, consistently balances prediction accuracy and computational efficiency throughout the training, validation, and testing phases. This ensures robust performance in real-world deployment scenarios.

VII. CONCLUSION AND FUTURE DIRECTION

Learning complex non-linear spatiotemporal dynamics in real-time traffic parameters can significantly improve model prediction performance. This research enhanced the accuracy and computational efficiency of RNN variants for the PeMS-10 real-time traffic dataset through two key innovations. First, we selected a near-optimal subset of features by integrating a genetic algorithm (GA) with random forest cross-validation. This novel combination reduces computational complexity and improves model prediction performance by reducing dimensionality while preserving the most important relationships between variables. Secondly, hyperparameter optimization is accomplished using a multi-objective NSGA-II-based RNN model, which balances the complexity of RNN topology with learning capability across different time-step sequence lengths. The combined approaches above result in a lightweight, more generalized model less susceptible to overfitting. An extensive experimental comparison study demonstrates the effectiveness of our approach. We achieved an R^2 score of 0.999490, with testing times of 180 milliseconds and a model size of 0.194 MB. Our proposed method supports ITS by predicting an accurate future traffic speed to optimize routes and reduce environmental impact. However, in future work, we will evaluate our approach using other benchmark datasets to improve generalization and robustness. Additionally, integrating a quantum computing-based approach could reduce the computational overhead of the conventional genetic algorithm.

REFERENCES

- [1] P. Dutta, S. Khatua, and S. Choudhury, "DB-corouting: Density based coordinated vehicle rerouting in smart environment," *Int. J. Intell. Transp. Syst. Res.*, vol. 19, no. 3, pp. 539–556, Sep. 2021, doi: [10.1007/s13177-021-00261-6](#).
- [2] TexasA&M TTI. (2023). *Urban Mobility*. [Online]. Available: <https://static.tti.tamu.edu/tti.tamu.edu/documents/mobility-report-2023-appx-b.pdf>
- [3] Y. Zhang and D. Xin, "A diverse ensemble deep learning method for short-term traffic flow prediction based on spatiotemporal correlations," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 16715–16727, Sep. 2022, doi: [10.1109/TITS.2021.3131248](#).
- [4] H. Zheng, F. Lin, X. Feng, and Y. Chen, "A hybrid deep learning model with attention-based conv-LSTM networks for short-term traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 6910–6920, Nov. 2021, doi: [10.1109/TITS.2020.2997352](#).
- [5] S. Lee and D. B. Fambro, "Application of subset autoregressive integrated moving average model for short-term freeway traffic volume forecasting," *Transp. Res. Record, J. Transp. Res. Board*, vol. 1678, no. 1, pp. 179–188, Jan. 1999, doi: [10.3141/1678-22](#).
- [6] B. M. Williams, "Multivariate vehicular traffic flow prediction: Evaluation of ARIMAX modeling," *Transp. Res. Record: J. Transp. Res. Board*, vol. 1776, no. 1, pp. 194–200, Jan. 2001, doi: [10.3141/1776-25](#).
- [7] B. M. Williams and L. A. Hoel, "Modeling and forecasting vehicular traffic flow as a seasonal ARIMA process: Theoretical basis and empirical results," *J. Transp. Eng.*, vol. 129, no. 6, pp. 664–672, Nov. 2003, doi: [10.1061/\(asce\)0733-947x\(2003\)129:6\(664\)](#).
- [8] X. Feng, X. Ling, H. Zheng, Z. Chen, and Y. Xu, "Adaptive multi-kernel SVM with spatial-temporal correlation for short-term traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 6, pp. 2001–2013, Jun. 2019.
- [9] G. Lin, A. Lin, and D. Gu, "Using support vector regression and K-nearest neighbors for short-term traffic flow prediction based on maximal information coefficient," *Inf. Sci.*, vol. 608, pp. 517–531, Aug. 2022, doi: [10.1016/j.ins.2022.06.090](#).
- [10] W. Shu, K. Cai, and N. N. Xiong, "A short-term traffic flow prediction model based on an improved gate recurrent unit neural network," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 16654–16665, Sep. 2022, doi: [10.1109/TITS.2021.3094659](#).
- [11] C. Ma, Y. Zhao, G. Dai, X. Xu, and S.-C. Wong, "A novel STFS-CNN-GRU hybrid model for short-term traffic speed prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3728–3737, Apr. 2023, doi: [10.1109/TITS.2021.3117835](#).
- [12] Q. Lai, J. Tian, W. Wang, and X. Hu, "Spatial-temporal attention graph convolution network on edge cloud for traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4565–4576, Apr. 2023, doi: [10.1109/TITS.2022.3185503](#).
- [13] D. Wu, Q. Guan, Z. Fan, H. Deng, and T. Wu, "AutoML with parallel genetic algorithm for fast hyperparameters optimization in efficient IoT time series prediction," *IEEE Trans. Ind. Informat.*, vol. 19, no. 9, pp. 9555–9564, Sep. 2023, doi: [10.1109/TII.2022.3231419](#).
- [14] Z. Chen, T. Lin, N. Tang, and X. Xia, "A parallel genetic algorithm based feature selection and parameter optimization for support vector machine," *Sci. Program.*, vol. 2016, no. 1, pp. 1–10, 2016.
- [15] C. Guo, L. Li, Y. Hu, and J. Yan, "A deep learning based fault diagnosis method with hyperparameter optimization by using parallel computing," *IEEE Access*, vol. 8, pp. 131248–131256, 2020, doi: [10.1109/ACCESS.2020.3009644](#).
- [16] X. Xiao, M. Yan, S. Basodi, C. Ji, and Y. Pan, "Efficient hyperparameter optimization in deep learning using a variable length genetic algorithm," 2020, *arXiv:2006.12703*.
- [17] B. Ding, Z. Rao, W. Yin, Y. Liu, J. Fang, Y. Wang, and P. Jin, "The optimization of urban traffic routes using an enhanced genetic algorithm: A case study of Beijing south railway station," *Appl. Sci.*, vol. 14, no. 14, p. 6130, Jul. 2024, doi: [10.3390/app14146130](#).
- [18] M. Farsi, D. Hosahalli, B. R. Manjunatha, I. Gad, E.-S. Atlam, A. Ahmed, G. Elmarhomy, M. Elmarhoumy, and O. A. Ghoneim, "Parallel genetic algorithms for optimizing the SARIMA model for better forecasting of the NCDC weather data," *Alexandria Eng. J.*, vol. 60, no. 1, pp. 1299–1316, Feb. 2021.
- [19] L. Guo, H. Liu, L. Han, C. Xiang, R. Liu, and N. Yang, "Co-optimization of speed planning and power management for automated HEVs in unstructured road scenarios," *IEEE Trans. Transport. Electric.*, vol. 10, no. 1, pp. 1628–1641, Mar. 2024, doi: [10.1109/TTE.2023.3281274](#).
- [20] A. S. Akopov, L. A. Beklaryan, and M. Thakur, "Improvement of maneuverability within a multiagent fuzzy transportation system with the use of parallel biobjective real-coded genetic algorithm," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 12648–12664, Aug. 2022, doi: [10.1109/TITS.2021.3115827](#).
- [21] L. C. Edie, "Car-following and steady-state theory for noncongested traffic," *Operations Res.*, vol. 9, no. 1, pp. 66–76, Feb. 1961, doi: [10.1287/opre.9.1.66](#).
- [22] G. E. P. Box, G. C. Reinsel, G. M. Jenkins, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Hoboken, NJ, USA: Wiley, 2015.
- [23] Y. Miao, X. Bai, Y. Cao, Y. Liu, F. Dai, F. Wang, L. Qi, and W. Dou, "A novel short-term traffic prediction model based on SVD and ARIMA with blockchain in industrial Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21217–21226, Dec. 15, 2023, doi: [10.1109/IIOT.2023.3283611](#).
- [24] W. Zheng, D. Lee, and Q. Shi, "Short-term freeway traffic flow prediction: Bayesian combined neural network approach," *J. Transp. Eng.*, vol. 132, no. 2, pp. 114–121, Jan. 2006, doi: [10.1061/\(ASCE\)0733-947X\(2006\)132:2\(114\)](#).
- [25] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, "Traffic flow prediction with big data: A deep learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 865–873, Apr. 2015, doi: [10.1109/TITS.2014.2345663](#).
- [26] Y. Duan, Y. Lv, Y.-L. Liu, and F.-Y. Wang, "An efficient realization of deep learning for traffic data imputation," *Transp. Res. C, Emerg. Technol.*, vol. 72, pp. 168–181, Nov. 2016, doi: [10.1016/j.trc.2016.09.015](#).
- [27] H.-F. Yang, T. S. Dillon, and Y. P. Chen, "Optimized structure of the traffic flow forecasting model with a deep learning approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2371–2381, Oct. 2017, doi: [10.1109/TNNLS.2016.2574840](#).
- [28] Z. Zhao, W. Chen, X. Wu, P. C. Y. Chen, and J. Liu, "LSTM network: A deep learning approach for short-term traffic forecast," *IET Intell. Transp. Syst.*, vol. 11, no. 2, pp. 68–75, Mar. 2017.

- [29] Y. Jia, J. Wu, M. Ben-Akiva, R. Seshadri, and Y. Du, "Rainfall-integrated traffic speed prediction using deep learning method," *IET Intell. Transp. Syst.*, vol. 11, no. 9, pp. 531–536, Nov. 2017.
- [30] Z. Abbas, A. Al-Shishtawy, Š. Girdzijauskas, and V. Vlassov, "Short-term traffic prediction using long Short-term memory neural networks," in *Proc. IEEE Int. Congr. Big Data (BigData Congress)*, Jul. 2018, pp. 57–65.
- [31] D. Zhang and M. R. Kabuka, "Combining weather condition data to predict traffic flow: A GRU-based deep learning approach," *IET Intell. Transp. Syst.*, vol. 12, no. 7, pp. 578–585, Sep. 2018.
- [32] Z. Cui, R. Ke, Z. Pu, and Y. Wang, "Deep bidirectional and unidirectional LSTM recurrent neural network for network-wide traffic speed prediction," 2018, *arXiv:1801.02143*.
- [33] J. Zhang, Y. Zheng, D. Qi, R. Li, X. Yi, and T. Li, "Predicting citywide crowd flows using deep spatio-temporal residual networks," *Artif. Intell.*, vol. 259, pp. 147–166, Jun. 2018.
- [34] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *Proc. 31st AAAI Conf. Artif. Intell.*, vol. 31, Feb. 2017, pp. 1655–1661.
- [35] H. Yu, Z. Wu, S. Wang, Y. Wang, and X. Ma, "Spatiotemporal recurrent convolutional networks for traffic prediction in transportation networks," *Sensors*, vol. 17, no. 7, p. 1501, Jun. 2017, doi: [10.3390/s17071501](https://doi.org/10.3390/s17071501).
- [36] G. Yang, Y. Wang, H. Yu, Y. Ren, and J. Xie, "Short-term traffic state prediction based on the spatiotemporal features of critical road sections," *Sensors*, vol. 18, no. 7, p. 2287, Jul. 2018, doi: [10.3390/s18072287](https://doi.org/10.3390/s18072287).
- [37] X. Ma, H. Zhong, Y. Li, J. Ma, Z. Cui, and Y. Wang, "Forecasting transportation network speed using deep capsule networks with nested LSTM models," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 4813–4824, Aug. 2021, doi: [10.1109/TITS.2020.2984813](https://doi.org/10.1109/TITS.2020.2984813).
- [38] Z. Cheng, J. Lu, H. Zhou, Y. Zhang, and L. Zhang, "Short-term traffic flow prediction: An integrated method of econometrics and hybrid deep learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 5231–5244, Jun. 2022, doi: [10.1109/TITS.2021.3052796](https://doi.org/10.1109/TITS.2021.3052796).
- [39] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," 2017, *arXiv:1707.01926*.
- [40] Z. Cui, K. Henrickson, R. Ke, and Y. Wang, "Traffic graph convolutional recurrent neural network: A deep learning framework for network-scale traffic learning and forecasting," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 11, pp. 4883–4894, Nov. 2020.
- [41] Y. Zhang, T. Cheng, Y. Ren, and K. Xie, "A novel residual graph convolution deep learning model for short-term network-based traffic forecasting," *Int. J. Geographical Inf. Sci.*, vol. 34, no. 5, pp. 969–995, May 2020.
- [42] K. Lee and W. Rhee, "DDP-GCN: Multi-graph convolutional network for spatiotemporal traffic forecasting," 2019, *arXiv:1905.12256*.
- [43] Z. Zhang, M. Li, X. Lin, Y. Wang, and F. He, "Multistep speed prediction on traffic networks: A deep learning approach considering spatio-temporal dependencies," *Transp. Res. C, Emerg. Technol.*, vol. 105, pp. 297–322, Aug. 2019.
- [44] Y. Shin and Y. Yoon, "Incorporating dynamicity of transportation network with multi-weight traffic graph convolutional network for traffic forecasting," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2028–2092, Mar. 26, 2022, doi: [10.1109/TITS.2020.3031331](https://doi.org/10.1109/TITS.2020.3031331).
- [45] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," 2017, *arXiv:1709.04875*.
- [46] D. E. Puentes G., C. J. Barrios H., and P. O. A. Navaux, "Hyper-parameter optimization for convolutional neural networks with genetic algorithms and Bayesian optimization," in *Proc. IEEE Latin Amer. Conf. Comput. Intell. (LA-CCI)*, Montevideo, Uruguay, Nov. 2022, pp. 1–5, doi: [10.1109/LA-CCI54402.2022.9981104](https://doi.org/10.1109/LA-CCI54402.2022.9981104).
- [47] T. Mao, A.-S. Mihaita, F. Chen, and H. L. Vu, "Boosted genetic algorithm using machine learning for traffic control optimization," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 7112–7141, Jul. 2022, doi: [10.1109/TITS.2021.3066958](https://doi.org/10.1109/TITS.2021.3066958).
- [48] L. Zhang, N. R. Alharbe, G. Luo, Z. Yao, and Y. Li, "A hybrid forecasting framework based on support vector regression with a modified genetic algorithm and a random forest for traffic flow prediction," *Tsinghua Sci. Technol.*, vol. 23, no. 4, pp. 479–492, Aug. 2018, doi: [10.26599/TST.2018.9010045](https://doi.org/10.26599/TST.2018.9010045).
- [49] A. Sadeghi-Niaraki, P. Mirshafiei, M. Shakeri, and S.-M. Choi, "Short-term traffic flow prediction using the modified Elman recurrent neural network optimized through a genetic algorithm," *IEEE Access*, vol. 8, pp. 217526–217540, 2020, doi: [10.1109/ACCESS.2020.3039410](https://doi.org/10.1109/ACCESS.2020.3039410).
- [50] California Dept. Transp. (2024). *Performance Measurement System (PeMS)*. [Online]. Available: <https://pems.dot.ca.gov/>



C. SWETHA PRIYA received the B.Tech. degree in computer science and engineering from JNTU, Anantapur, and the M.Tech. degree in information security from Pondicherry University. She is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering, Puducherry Technological University, India. She has qualified for both UGC-NET and APSET for assistant professorships. She has published seven research papers in international journals and national and international conferences. Her research interests include deep learning, the Internet of Things (IoT), and traffic data analysis.



F. SAGAYARAJ FRANCIS received the Ph.D. degree in computer science and engineering from Pondicherry University, Puducherry, India, in 2008. He is currently a Professor with the Department of Computer Science and Engineering, Puducherry Technological University, Puducherry. He has authored or co-authored over 60 publications. His research interest includes data science.

...